

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

I G Amrutha(192511250) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

amrutha

Annexure A

Reverse Engineering

1. Reverse Engineering a Recommendation System

A large-scale e-commerce platform provides highly accurate product recommendations using real-time data. Analyze the system and reverse engineer its architecture using PySpark and RDD operations. Identify how data is collected, processed, and transformed, and infer the role of intelligent agents in updating recommendations dynamically.

2. Reverse Engineering a Smart Traffic Management System

A city-wide traffic system adapts signals in real time based on vehicle density and congestion patterns. Reverse engineer the underlying AI system by describing the data pipeline using RDD-based streaming, identifying agent interactions, and explaining how decisions are made under time constraints.

3. Reverse Engineering a Fraud Detection Pipeline

A banking system flags fraudulent transactions within milliseconds. By observing its outputs and behavior, reconstruct the distributed AI pipeline using PySpark RDDs. Explain how feature extraction, anomaly detection, and agent-based alert mechanisms are likely implemented.

4. Reverse Engineering a Personalized Learning System

An online learning platform adapts content difficulty based on student performance and engagement. Reverse engineer the system design by identifying how learner data is processed using RDD transformations, and explain how intelligent agents contribute to adaptive learning and feedback mechanisms.

5. Reverse Engineering a Cloud-Based AI Monitoring System

A cloud platform automatically scales resources and detects system anomalies in real time. Reverse engineer the architecture by analyzing how PySpark processes large-scale logs, how RDD operations support fault tolerance, and how intelligent agents coordinate monitoring, scaling, and anomaly response.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

For this Reverse Engineering rubric, I recommend Topic 1 – Reverse Engineering a Recommendation System.

It is the strongest choice because it naturally lets you demonstrate real-world industry understanding, PySpark/RDD operations, AI recommendation logic, intelligent agents, architecture reconstruction, analysis, and performance evaluation.

Best topic

Reverse Engineering a Large-Scale E-Commerce Recommendation System Using PySpark and Intelligent Agents

Why Topic 1 is best

Rubric	Topic 1 advantage
Industry Problem – 20%	E-commerce recommendation is a very clear real-world problem
Engineering Concepts – 25%	RDDs, distributed processing, filtering, aggregation, similarity and ranking
Solution Design – 25%	Easy to design a complete recommendation architecture
Tools – 15%	Python, PySpark, RDDs, Jupyter/Colab and visualization
Analysis – 10%	Can compare recommendation accuracy, response time and scalability
Professionalism – 5%	Easy to present as an industry-style reverse-engineering case study

Suggested presentation structure

Slide 1 – Title

Reverse Engineering a Large-Scale E-Commerce Recommendation System Using PySpark and Intelligent Agents

Subtitle:

Architecture Reconstruction, Distributed Data Processing and Dynamic Recommendation

Slide 2 – Industry Problem

An e-commerce platform may receive millions of:

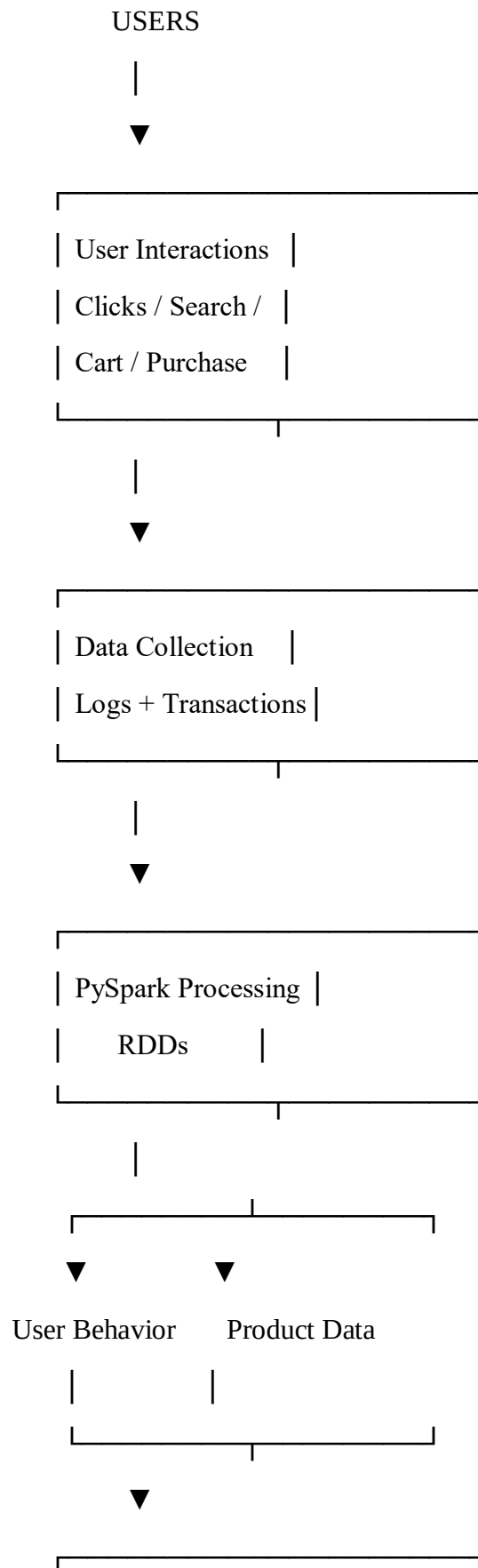
- User searches
- Product views
- Clicks
- Purchases
- Ratings
- Cart additions
- Wishlist actions

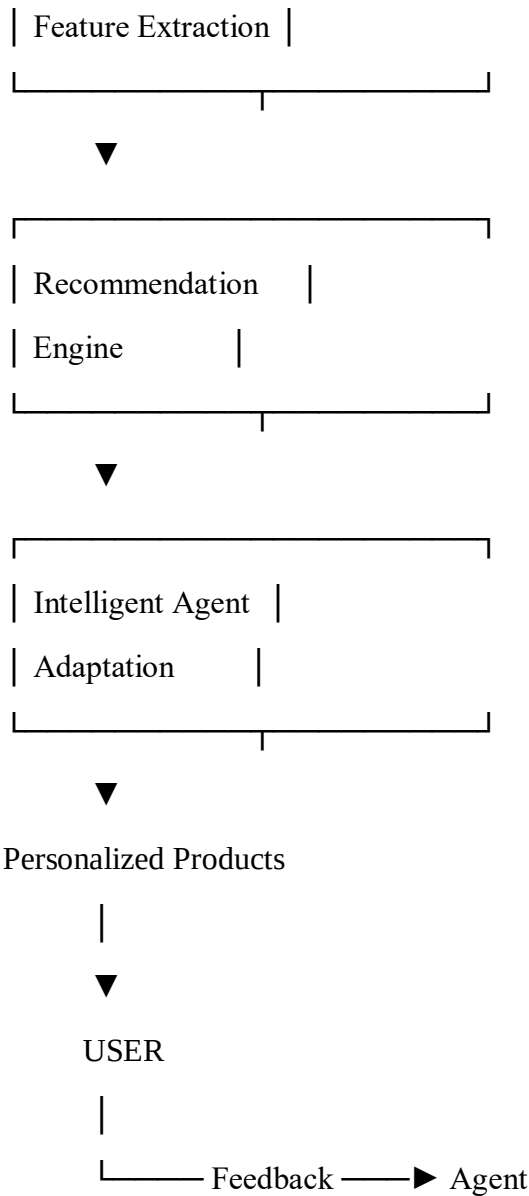
The system must convert these interactions into personalized recommendations in near real time.

Reverse-engineering objective:

Infer how the hidden recommendation system collects data, processes it, generates recommendations, and dynamically adapts to user behavior.

Slide 3 – Reverse-Engineered Architecture





Slide 4 – Data Collection

The reconstructed system would likely collect:

Data	Example
User ID	U101
Product ID	P501
Action	Click
Rating	4/5
Search	"wireless headphones"
Purchase	Yes

Data Example

Timestamp 10:35 AM

Session S1001

The system combines behavioral data + product information to understand user preferences.

Slide 5 – PySpark RDD Processing

Example:

```
data = sc.textFile("user_activity.csv")
```

```
rdd = data.map(lambda x: x.split(","))
```

Filter invalid records:

```
valid_data = rdd.filter(  
    lambda x: len(x) >= 4  
)
```

Create user-product pairs:

```
user_products = valid_data.map(  
    lambda x: (x[0], x[1])  
)
```

Count interactions:

```
interaction_count = user_products.map(  
    lambda x: (x, 1)  
)  
.reduceByKey(  
    lambda a, b: a + b  
)
```

This demonstrates important RDD operations:

```
map() → filter() → reduceByKey() → groupByKey() → join() → sortBy()
```

Slide 6 – Feature Extraction

User behavior can be converted into numerical features.

For example:

Click = 1

Wishlist = 3

Cart = 5

Purchase = 10

Rating 5 = 8

Example:

User U101

Product A → Click + Cart

Product B → Click

Product C → Purchase

Preference Score:

Product C = Highest

Product A = Medium

Product B = Low

This allows the hidden recommendation logic to be reconstructed as a behavior-scoring mechanism.

Slide 7 – Recommendation Logic

A possible reconstructed scoring formula:

Recommendation Score =

$0.4 \times \text{User Preference}$

- $0.3 \times \text{Product Similarity}$
- $0.2 \times \text{Popularity}$
- $0.1 \times \text{Recent Activity}$

Example:

User Preference = 0.90

Product Similarity = 0.80

Popularity = 0.70

Recent Activity = 0.85

Therefore:

Score =

$(0.4 \times 0.90) +$

$(0.3 \times 0.80) +$

$(0.2 \times 0.70) +$

(0.1×0.85)

$= 0.36 + 0.24 + 0.14 + 0.085$

$= 0.825$

Final recommendation score = 0.825

Slide 8 – Intelligent Agents

Use several agents in the reconstructed architecture:

User Behavior Agent

- Monitors clicks, searches and purchases.
- Updates the user profile.

Recommendation Agent

- Generates and ranks candidate products.

Feedback Agent

- Observes whether users click, purchase or ignore recommendations.

Adaptation Agent

- Changes recommendation priorities based on new behavior.

New User Action



Behavior Agent



Update User Profile



Recommendation Agent



Generate New Products



Feedback Agent



Adaptation Agent



Improved Recommendation

Slide 9 – Reverse Engineering Method

You can explain the investigation as:

Observe System Output



Identify Input Variables



Study Recommendation Changes



Infer Data Processing



Reconstruct RDD Pipeline



Infer Recommendation Logic



Model Intelligent Agents



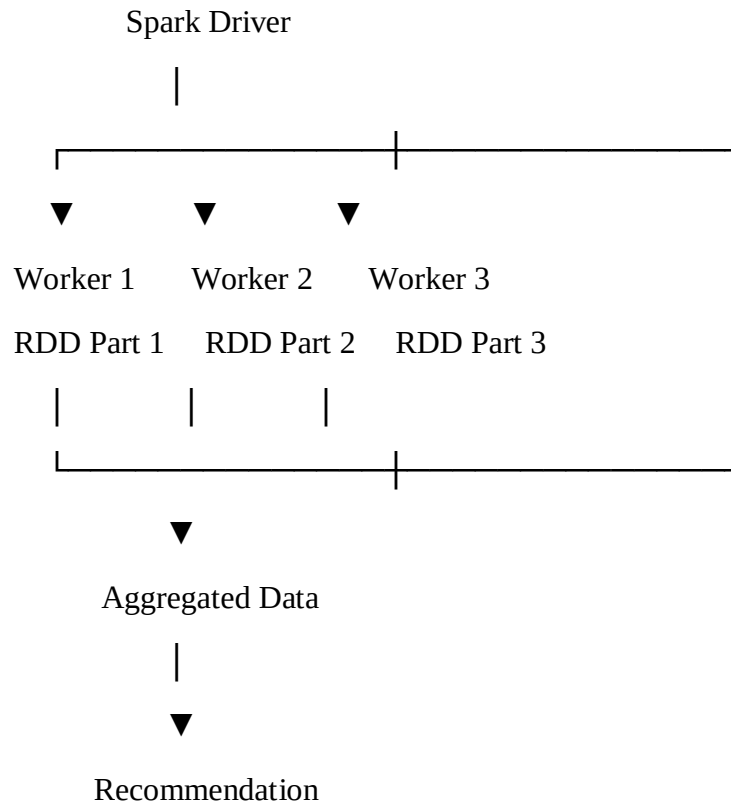
Implement Prototype



Compare Predicted vs Observed Results

This is important because your topic is reverse engineering, not simply designing a recommendation system.

Slide 10 – Distributed Processing



Explain:

- RDDs divide large datasets into partitions.
- Different workers process partitions in parallel.
- `reduceByKey()` performs distributed aggregation.
- RDD lineage provides fault recovery.
- More worker nodes can be added for larger datasets.

Slide 11 – Analysis and Performance Evaluation

This directly targets the 10% Analysis criterion.

Compare:

Metric	Traditional System	Reverse-Engineered System
Processing time	Higher	Lower
Scalability	Limited	High
Personalization	Moderate	High
Real-time adaptation	Limited	High

Metric	Traditional System	Reverse-Engineered System
Fault tolerance	Moderate	High

You can also evaluate:

- Recommendation accuracy
- Precision
- Recall
- Processing time
- Throughput
- Scalability
- User engagement

Slide 12 – Example Result

User: U101

Recent Activity:

- ✓ Viewed laptops
- ✓ Searched gaming laptops
- ✓ Added laptop to cart
- ✓ Purchased wireless mouse

Agent Interpretation:

User has high interest in computer accessories.

Recommended Products:

1. Gaming Keyboard 0.94
2. Gaming Headset 0.91
3. Laptop Cooling Pad 0.87
4. Gaming Monitor 0.84

Slide 13 – Key Findings

The reverse-engineered system is likely based on:

Large-scale behavioral data

↓

Distributed PySpark processing

↓

Feature extraction

↓

Recommendation scoring

↓

Ranking

↓

Intelligent-agent adaptation

↓

Personalized recommendations

Slide 14 – Innovation

Add an Adaptive Recommendation Agent.

Instead of using fixed weights:

Content = 50%

Behavior = 50%

the agent dynamically changes them.

For a new user:

Content = 70%

Behavior = 30%

For an experienced user:

Content = 30%

Behavior = 70%

This solves the cold-start problem while allowing the system to become more personalized over time.

Slide 15 – Conclusion

The reverse-engineered recommendation system demonstrates how a large-scale e-commerce platform can combine distributed PySpark RDD processing, behavioral feature extraction, recommendation scoring, and intelligent-agent adaptation.

The reconstructed architecture provides:

- Scalable processing
- Personalized recommendations
- Dynamic adaptation
- Fault-tolerant distributed computation
- Continuous learning from user feedback

☆ My ranking of the five topics

1. Recommendation System ☆ ☆ ☆ ☆ ☆ — Best overall
2. Fraud Detection Pipeline ☆ ☆ ☆ ☆ ☆ — Very strong technically, but slightly harder
3. Smart Traffic Management ☆ ☆ ☆ ☆ ☆ — Excellent real-time/agent topic
4. Cloud AI Monitoring ☆ ☆ ☆ ☆ ☆ — Strong distributed-systems topic
5. Personalized Learning ☆ ☆ ☆ ☆ ☆ — Easiest to explain, but less challenging for reverse engineering